

RAG Pipeline

Technical Design Document

Retrieval-Augmented Generation (RAG) Document Q&A System

Version 2.0

Prepared by Shubhi Agarwal

1. High-Level Design (HLD)

1.1 System Overview

The RAG Pipeline is a Retrieval-Augmented Generation (RAG) application that enables users to upload PDF documents, generate vector embeddings, and query those documents through a conversational chat interface powered by a Large Language Model (LLM). Rather than relying on the model's training data, the system retrieves the most relevant passages from the user's own documents and grounds every answer in that retrieved context, returning cited, source-attributed responses.

The system is organised into two primary flows and a supporting evaluation layer:

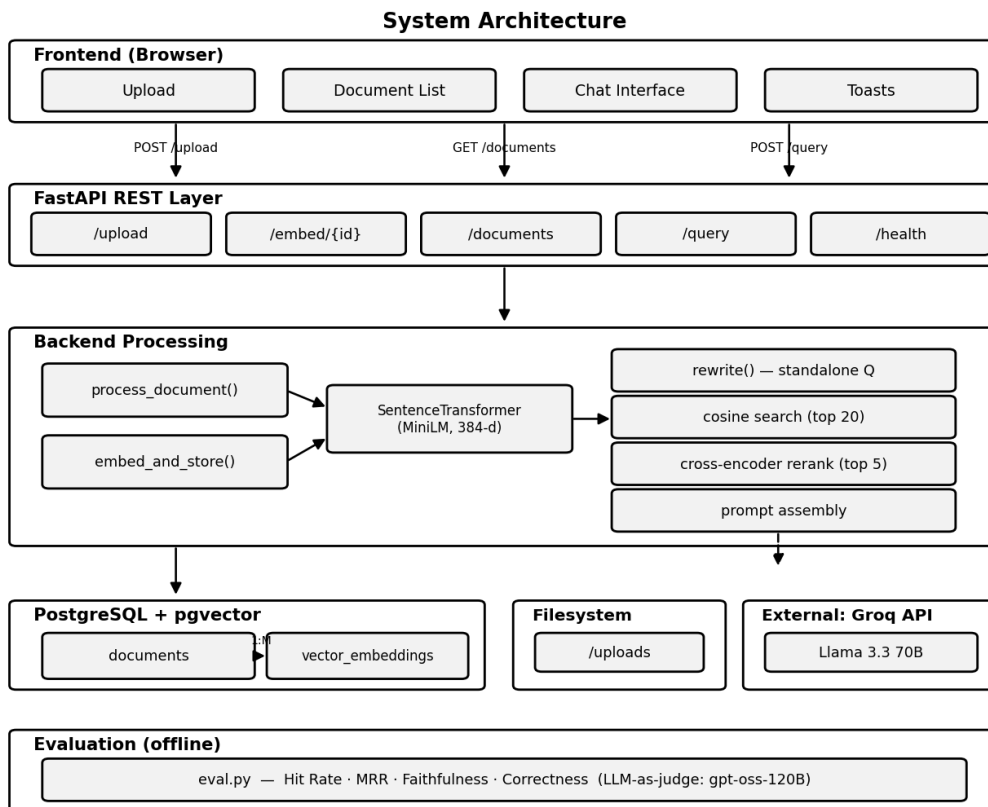
- Ingestion: PDF upload, text extraction and cleaning, chunking, keyword enrichment, and vector embedding generation.
- Retrieval & Generation: conversational query rewriting, two-stage retrieval (bi-encoder similarity search followed by cross-encoder reranking), context assembly, and LLM answer generation with source citations.
- Evaluation (offline): an LLM-as-judge framework that scores retrieval and answer quality against a labelled test set.

1.2 System Architecture

The architecture is layered: a browser-based frontend, a FastAPI REST layer, a backend processing layer containing the core functions, a PostgreSQL/pgvector data store, the local filesystem for uploaded PDFs, and the external Groq API for LLM inference. The conversation history is held client-side and submitted with each query, keeping the backend stateless.

Architecture Layers

Layer	Responsibility
Frontend (Browser)	Upload UI, document list, chat interface, toasts; holds conversation history
FastAPI REST Layer	Endpoints: /upload, /embed/{id}, /documents, /query, /health
Backend Processing	process_document(), embed_and_store(), rewrite(), query()
Data Store	PostgreSQL + pgvector: document metadata and 384-dim vector embeddings
Filesystem	Stored PDF documents under /uploads
External Service	Groq API for LLM generation and LLM-as-judge evaluation



1.3 Components

- Ingestion pipeline — extracts and cleans PDF text, removes repeated headers and footers, chunks the text with overlap, enriches chunks with TF-IDF keywords, and stores embeddings.
- Retrieval engine — embeds the query, performs cosine similarity search over pgvector, and reranks candidates with a cross-encoder.
- Conversational layer — rewrites follow-up questions into standalone questions using conversation history before retrieval.
- Generation layer — assembles retrieved context into a constrained prompt and calls the LLM, returning a grounded, cited answer.
- Evaluation framework — an offline pipeline measuring Hit Rate, MRR, Faithfulness, and Correctness.

1.4 External Integrations

- Groq API — cloud-hosted LLM inference for answer generation (Llama 3.3 70B), query rewriting, and the evaluation judge (gpt-oss-120B). Requires an API key supplied via the `GROQ_API_KEY` environment variable.
- SentenceTransformers / CrossEncoder — embedding and reranking models downloaded once at startup; run locally with no external calls during inference.
- No other external services are required; the data store is a local PostgreSQL instance.

1.5 Tech Stack

Layer	Technology	Purpose
Language	Python 3.11+	Backend logic and ML model integration
Backend	FastAPI	Async REST API with auto-generated Swagger docs
Embedding (bi-encoder)	all-MiniLM-L6-v2	384-dim sentence embeddings, runs locally
Reranker (cross-encoder)	ms-marco-MiniLM-L-6-v2	Relevance reranking of retrieval

Layer	Technology	Purpose
		candidates
Vector DB	PostgreSQL + pgvector	Cosine similarity search alongside metadata
LLM (generation/rewrite)	Groq (Llama 3.3 70B)	Fast inference for answers and query rewriting
LLM (evaluation judge)	Groq (gpt-oss-120B)	Offline scoring of answer quality
PDF Processing	pdfplumber	Text extraction from PDF documents
Frontend	HTML / CSS / JavaScript	Single-page interface with drag-drop upload
Validation	Pydantic	Request/response model validation

2. Low-Level Design (LLD)

2.1 Core Functions

Function 1: *process_document(file_path)*

Responsibility: extract text from a PDF and split it into overlapping, keyword-enriched chunks.

- Input: `file_path` (str) — path to the PDF on disk
- Output: list of dictionaries, each with `page_no`, `chunk_index`, `text`, and `keywords`

Process:

- Open the PDF with `pdfplumber` and extract text page by page, skipping pages that return no text.
- Detect and remove repeated headers, footers, and page numbers — only when the document has 3 or more pages, to avoid stripping all text from single-page documents.
- Normalise whitespace, collapsing tabs, newlines, and repeated spaces into single spaces.
- Chunk the text with a sliding window: 400-character step over a 500-character window (100-character overlap).
- Extract the top keywords per chunk using TF-IDF and append them to the chunk text to enrich the embedding.

Function 2: *embed_and_store(document_id, chunks)*

Responsibility: convert text chunks to vectors and persist them in `pgvector`.

- Input: `document_id` (int), `chunks` (list of dicts from Function 1)
- Output: none (side effect: rows inserted into `vector_embeddings`)

Process:

- Open a database connection with the `pgvector` type registered.
- For each chunk, encode the chunk text plus its keywords using `SentenceTransformer.encode()`.
- Build a structured `chunk_id` (e.g. `doc1_p3_c2`) and a JSON metadata object (page number, chunk index, keywords).
- Insert `document_id`, `chunk_id`, `text`, `metadata`, and `vector`; commit in a single transaction after the loop.

Function 3: *rewrite(conversation_history, question)*

Responsibility: rewrite a follow-up question into a self-contained standalone question.

- Input: `conversation_history` (list of prior turns), `question` (str)
- Output: standalone question (str)

Process:

- If the question depends on earlier context (e.g. pronouns such as “it”), use the history to rewrite it into a complete question.
- If the question is already self-contained or introduces a new topic, return it unchanged.
- Performed with Llama 3.3 70B; a smaller model was trialled but garbled multi-constraint rewrites.

Function 4: *query(question, document_ids)*

Responsibility: the full retrieval-and-generation pipeline.

- Input: `question` (str), `document_ids` (list of int)
- Output: answer string and the list of source chunks used

Process:

- Embed the (rewritten) question using the same `SentenceTransformer` model as the chunks.
- Run a cosine similarity search against `pgvector`, filtered by the selected `document_ids`, retrieving the top 20 candidates.
- Rerank the 20 candidates with the cross-encoder and keep the top 5.

- Re-sort the final 5 by document position (page, then chunk) so multi-part answers follow document order.
- If no chunks are found, return a fallback message.
- Assemble a labelled context string and a constrained prompt (answer only from context; clean prose with sources grouped at the end).
- Call the Groq API and return the answer with its source chunks.

2.2 Key Design Decisions

- Two-stage retrieval: a cheap bi-encoder narrows thousands of chunks to 20 candidates; an expensive but precise cross-encoder reranks those to the best 5.
- Document-order presentation: the final chunks are selected by relevance but presented to the LLM in document order, so multi-part answers read in sequence.
- Stateless conversational history: the browser holds conversation state and sends it per request; follow-ups are rewritten to standalone questions before retrieval.
- TF-IDF keyword enrichment: keywords are appended to chunks at embedding time so differently-worded questions still match the underlying concepts.
- Single embedding model instance: loaded once at startup to avoid reloading the model per request.
- Prompt engineering: the system prompt constrains the LLM to answer only from provided context, acknowledge conflicts, and admit when information is insufficient.
- Separate judge model: evaluation uses a different LLM from the generator to avoid self-evaluation bias.

2.3 Logic Flow — Sequence Diagrams

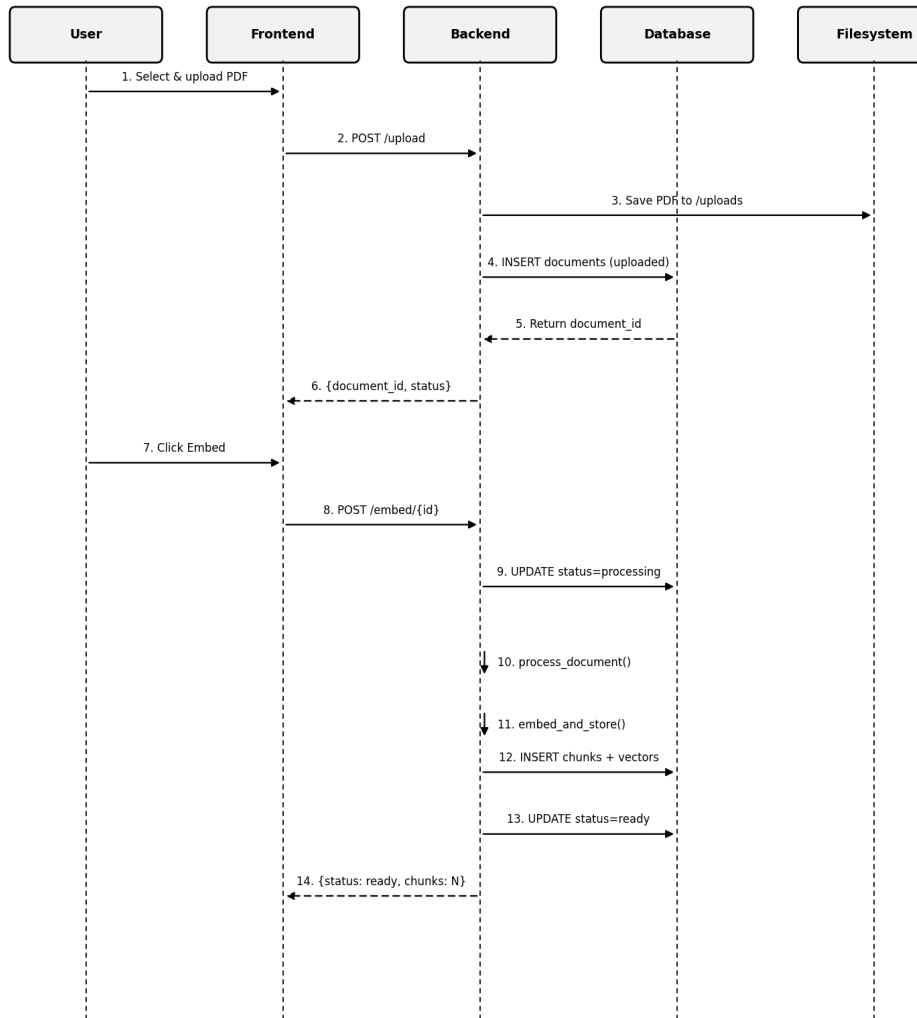
Document Upload & Embedding

- User selects and uploads a PDF; frontend sends POST /upload (multipart/form-data).
- Backend saves the PDF to /uploads and inserts a documents row with status “uploaded”; returns document_id.
- User clicks Embed; frontend sends POST /embed/{document_id}.
- Backend sets status “processing”, runs process_document() then embed_and_store(), then sets status “ready”.
- Frontend shows the READY badge and enables the document’s checkbox.

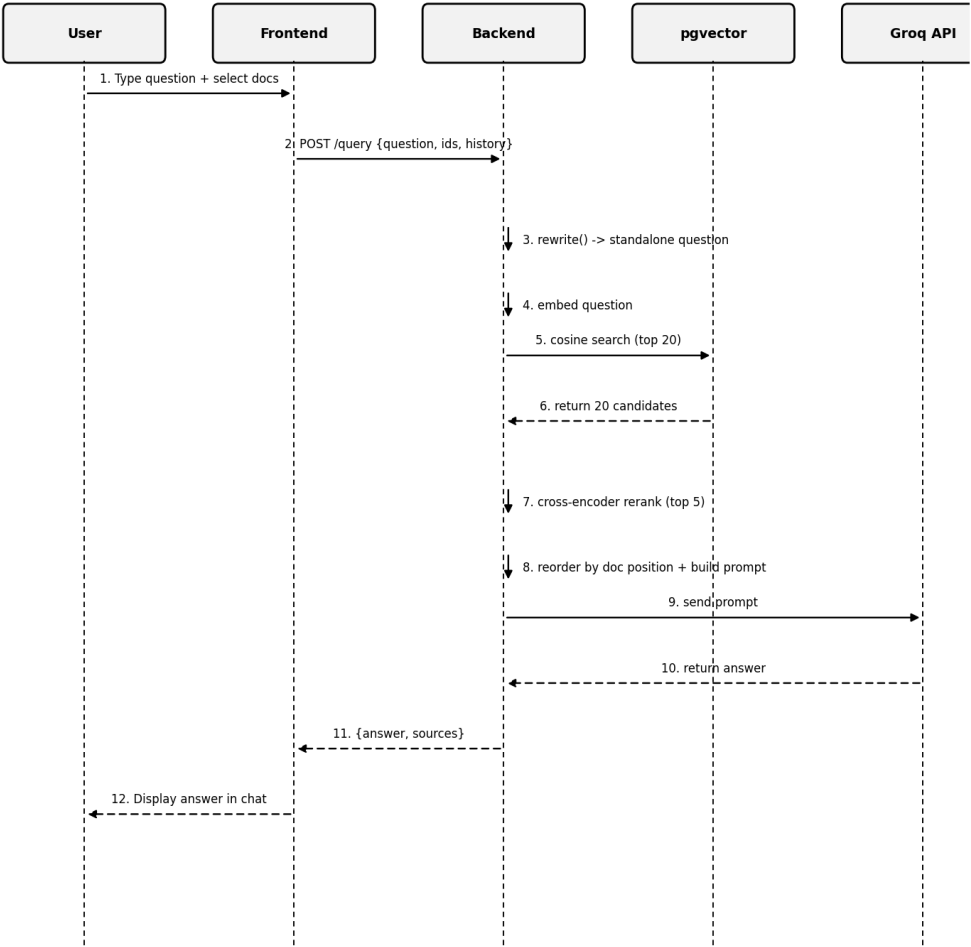
Query Pipeline

- User types a question and selects documents; frontend sends POST /query with question, document_ids, and conversation history.
- Backend rewrites the question to standalone form, embeds it, and runs cosine similarity search (top 20) against pgvector.
- Backend reranks to the top 5, reorders by document position, and assembles the prompt.
- Backend calls the Groq API and returns the generated answer with sources.
- Frontend displays the answer in the chat and appends the turn to conversation history.

Sequence: Document Upload & Embedding



Sequence: Query Pipeline



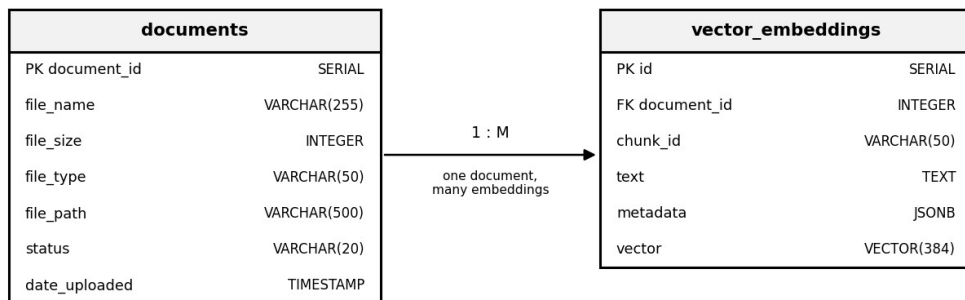
3. Data Model / Database Schema (ERD)

3.1 Entity Relationship Overview

The schema has two tables in a one-to-many relationship: one document has many vector embeddings. Each embedding references its parent document via a foreign key.

Relationship	Cardinality	Description
documents → vector_embeddings	1 : M	One document produces many embedded chunks

Entity Relationship Diagram



3.2 Data Dictionary: documents

Column	Type	Null	Description
document_id	SERIAL (PK)	No	Auto-incremented primary key
file_name	VARCHAR(255)	No	Original filename of the uploaded PDF
file_size	INTEGER	Yes	File size in bytes
file_type	VARCHAR(50)	Yes	File extension (e.g. 'pdf')
file_path	VARCHAR(500)	No	Path to the file on the server filesystem
status	VARCHAR(20)	Yes	uploaded processing ready failed
date_uploaded	TIMESTAMP	Yes	Defaults to CURRENT_TIMESTAMP

3.3 Data Dictionary: vector_embeddings

Column	Type	Null	Description
id	SERIAL (PK)	No	Auto-incremented primary key
document_id	INTEGER (FK)	Yes	References documents.document_id
chunk_id	VARCHAR(50)	Yes	Structured chunk identifier (e.g. doc1_p3_c2)
text	TEXT	No	Original chunk text used for LLM context

Column	Type	Null	Description
metadata	JSONB	Yes	Page number, chunk index, and keywords
vector	VECTOR(384)	Yes	384-dim embedding from all-MiniLM-L6-v2

4. API Contracts / Interface Specs

Base URL: <http://localhost:8000>. Auto-generated OpenAPI documentation is available at </docs> (Swagger UI).

4.1 GET /health

Field	Value
Method	GET
Response	{"status": "ok"}

4.2 POST /upload

Upload a PDF document. Saves the file to disk and creates a metadata record.

Field	Value
Content-Type	multipart/form-data
Request Body	file: UploadFile (PDF only)
Success (200)	{document_id, file_name, status: "uploaded"}
Error (400)	Non-PDF file uploaded

4.3 POST /embed/{document_id}

Trigger text extraction, chunking, enrichment, and embedding for a document.

Field	Value
Path Parameter	document_id (integer)
Success (200)	{document_id, status: "ready", chunks_created: N}
Error (404)	Document ID not found
Error (500)	Processing or embedding failure

4.4 GET /documents

Field	Value
Success (200)	{documents: [{document_id, file_name, status, ...}]}

4.5 POST /query

Submit a question against selected documents. Accepts conversation history for follow-ups and returns an LLM-generated, cited answer.

Field	Value
Content-Type	application/json
Request Body	{question, document_ids: [...], history: [...]}
Success (200)	{answer, question, standalone_question}
Error (400)	Empty document_ids list

4.6 Error Response Format

All errors follow the standard FastAPI format: {"detail": "error message"}.

Code	Meaning	When
400	Bad Request	Invalid file type, missing fields, empty document_ids
404	Not Found	Document ID does not exist
500	Server Error	PDF processing, embedding, or LLM API failure

5. UI/UX Design

5.1 Wireframe

The interface is a single-page layout: a left sidebar for upload and the document list, and a main panel containing the chat header, message thread, and input bar. Selected documents are shown in the header, user messages are right-aligned, and assistant messages are left-aligned with sources grouped at the end of each answer.

UI Wireframe

The wireframe illustrates a single-page application layout. On the left is a sidebar titled "RAG Pipeline" with the subtitle "Upload PDFs, ask questions". It features a dashed box for dropping a PDF, a list of documents under "YOUR DOCUMENTS" (Report.pdf, Notes.pdf, Draft.pdf) with checkboxes and status badges (READY, READY, UPLOADED), and a "Send" button. The main panel has a header showing "2 documents selected for chat." and a message input area with a "Send" button. The message thread displays a user message "What are the steps?" and an assistant response "Based on the document, the steps are... (answer in order, sources grouped at end)".

5.2 Component Breakdown

Upload Dropzone

- Drag-and-drop zone with click-to-browse fallback.
- Accepts only .pdf files; rejects others with an error toast.
- Visual feedback on dragover (border colour change).

Document List

- Displays all documents sorted by upload date (newest first).
- Each item shows filename, size, status badge, and an action button.
- Status badges: UPLOADED, PROCESSING, READY, FAILED.
- Checkbox enabled only when status is READY; changing the selection starts a fresh conversation.

Chat Interface

- Header shows the count of selected documents.
- User messages right-aligned; assistant messages left-aligned with grouped sources.
- Input bar with text field and Send button; supports the Enter key.
- Thinking indicator shown during processing; conversation history maintained client-side.

Toast Notifications

- Bottom-right positioned; auto-dismiss after 4 seconds.
- Three variants: info, success, error.

5.3 Style Guide

Property	Value
Background (main)	#F7F8FA (light grey)
Background (sidebar)	#FFFFFF (white)
Text (primary)	#1F2330 (near-black)
Accent	#4F8FF7 (blue)
Font family	Segoe UI, -apple-system, sans-serif
Border radius	8-10px for cards, 16px for chat bubbles

6. Evaluation

An offline evaluation pipeline measures system quality on a labelled test set, so that changes are validated with metrics rather than assumption.

Metric	What it measures	Method
Hit Rate	Whether any retrieved chunk came from the correct page	Page match from chunk_id
MRR	How highly the first correct chunk was ranked	Reciprocal rank
Faithfulness	Whether the answer is grounded in retrieved chunks	LLM-as-judge (1–5)
Correctness	Whether the answer matches the expected answer	LLM-as-judge (1–5)

Faithfulness and Correctness are scored by a separate LLM (gpt-oss-120B) from the generator, using a structured 1–5 rubric that returns a score and a justification. Adding cross-encoder reranking improved Hit Rate (0.78 → 0.83) and Faithfulness (0.92 → 0.97), with a small dip in Correctness on multi-part questions — a relevance-versus-completeness trade-off identified by analysing the judge’s reasoning.

6.1 Known Limitations & Next Steps

- Generic queries across multiple documents can be dominated by a larger document; per-document candidate pools would address this.
- The sources list reflects retrieved chunks, not necessarily those used in the answer; per-claim attribution is a planned improvement.
- The embedding model truncates questions beyond 512 tokens.
- Scanned or image-only PDFs produce no extractable text; OCR support is a defined next step.
- A Groq API failure mid-query is not yet handled gracefully; a try/except with a user-facing message is planned.